

Расчет статистики столбца

Агрегатные функции суммируют множество точек данных (т. Е. Столбец фрейма данных) в меньший набор значений.

Некоторые примеры этого типа расчета включают в себя:

- Клиенты DataFrame содержат имена и возрасты всех ваших клиентов. Вы хотите найти средний возраст:

```
print(customers.age)
>> [23, 25, 31, 35, 35, 46, 62]
print(customers.age.median())
>> 35
```

DataFrame содержит адресную информацию для всех отправок, отправленных вами за последний год. Вы хотите знать, в сколько разных штатов вы отправили (и сколько поставок было отправлено в одно и то же место).

```
print(shipments.state)
>> ['CA', 'CA', 'CA', 'CA', 'NY', 'NY', 'NJ', 'NJ', 'NJ',
'NJ', 'NJ', 'NJ', 'NJ']
print(shipments.state.nunique())
>> 3
```

В инвентаре DataFrame содержится список типов футболок, которые производит ваша компания. Вам нужен список цветов ваших рубашек.

```
• print(inventory.color)
>> ['blue', 'blue', 'blue', 'blue', 'blue', 'green',
'green', 'orange', 'orange', 'orange']
print(inventory.color.unique())
>> ['blue', 'green', 'orange']
```

Общий синтаксис этих вычислений:

```
df.column_name.command()
```

В следующей таблице приведены некоторые общие команды

mean	Average of all values in column
std	Standard deviation
median	Median
max	Maximum value in column
min	Minimum value in column
count	Number of values in column
nunique	Number of unique values in column
unique	List of unique values in column

Задание

1. Мы еще раз вернемся к нашим заказам с ShoeFly.com. Наша новая партия заказов находится в заказах DataFrame. Изучите первые 10 строк, используя следующий код:

```
print(orders.head(10))
```

2. Наш финансовый отдел хочет знать цену самой дорогой купленной пары обуви. Сохраните свой ответ в переменной `most_expensive`.
3. Наш отдел моды хочет знать, сколько обуви разных цветов мы продаем. Сохраните свой ответ в переменной `num_colors`.

Вычисление агрегатных функций I

Когда у нас есть набор данных, мы часто хотим вычислить совокупную статистику (среднее значение, стандартное отклонение, медиана, процентиля и т. Д.) По определенным подмножествам данных.

Предположим, у нас есть зачетная книжка со столбцами "student", "assignment_name" и "оценка". Первые несколько строк выглядят так:

student	assignment_name	grade
Amy	Assignment 1	75
Amy	Assignment 2	35
Bob	Assignment 1	99
Bob	Assignment 2	35
...		

Мы хотим получить среднюю оценку за каждого студента по всем заданиям. Мы могли бы сделать какой-то цикл, но Pandas предлагает нам гораздо более простой вариант: метод `.groupby`.

В этом примере мы бы использовали следующую команду:

```
grades = df.groupby('student').grade.mean()
```

Результат может выглядеть примерно так:

student	grade
Amy	80
Bob	90
Chris	75
...	

Как правило, мы используем следующий синтаксис для вычисления агрегатов:

```
df.groupby('column1').column2.measurement()
```

где:

- `column1` — это столбец, который мы хотим сгруппировать по (в нашем примере "студент")
- `column2` — это столбец, в котором мы хотим провести измерение (оценка в нашем примере)
- `измерение` — это функция измерения, которую мы хотим применить (среднее в нашем примере)

Задание

1. Вернемся к нашим данным о заказах с ShoeFly.com.

В предыдущем упражнении наш финансовый отдел хотел узнать самую дорогую обувь, которую мы продали.

Теперь они хотят знать самую дорогую обувь для каждого типа обуви (т. Е. Самые дорогие ботинки, самые дорогие балетки и т. Д.).

Сохраните свой ответ в переменной `pricey_shoes`.

2. Изучите только что созданный объект, используя:

```
print(pricey_shoes)
```

3. Какой тип объекта `pricey_shoes`?

Введите следующий код для проверки:

```
print(type(pricey_shoes))
```

Расчет агрегатных функций II

После использования `groupby` нам часто нужно очистить полученные данные.

Как мы видели в предыдущем упражнении, функция `groupby` создает новую серию, а не `DataFrame`. В нашем примере ShoeFly.com индексами Серии были разные значения параметра `shoe_type`, а свойством `name` было `price`.

Обычно мы предпочитаем, чтобы эти индексы были фактически столбцами. Для этого мы можем использовать `reset_index()`. Это преобразует нашу серию в `DataFrame` и переместит индексы в их собственный столбец.

Как правило, вы всегда увидите оператор `groupby`, за которым следует `reset_index`:

```
df.groupby('column1').column2.measurement()  
    .reset_index()
```

Когда мы используем `groupby`, мы часто хотим переименовать столбец, который мы получаем в результате. Например, предположим, что у нас есть DataFrame `teas`, содержащий данные о типах чая:

id	tea	category	caffeine	price
0	earl grey	black	38	3
1	english breakfast	black	41	3
2	irish breakfast	black	37	2.5
3	jasmine	green	23	4.5
4	matcha	green	48	5
5	camomile	herbal	0	3
...				

Мы хотим найти количество каждой категории чая, который мы продаем. Мы можем использовать:

```
teas_counts = teas.groupby('category').id.count().reset_index()
```

Это дает DataFrame, который выглядит так:

	category	id
0	black	3
1	green	4
2	herbal	8
3	white	2
...		

Новый столбец содержит количество проданных сортов чая. У нас есть 3 черных чая, 4 зеленых чая и так далее. Однако этот столбец называется `id`, потому что мы использовали столбец `id` чаев для подсчета количества. На самом деле мы хотим назвать этот столбец счетчиками. Помните, что мы можем переименовывать столбцы:

```
teas_counts = teas_counts.rename(columns={"id": "counts"})
```

Теперь наш DataFrame выглядит так:

	category	counts
0	black	3
1	green	4
2	herbal	8
3	white	2
...		

Задание

1. Измените свой код из предыдущего упражнения так, чтобы он заканчивался на `reset_index`, который преобразует `pricey_shoes` в `DataFrame`.
2. Изучите только что созданный объект, используя следующий код:

```
print(pricey_shoes)
```

3. Теперь, какой тип объекта `pricey_shoes`?

Введите следующий код для проверки:

```
print(type(pricey_shoes))
```

```
import pandas as pd

orders = pd.read_csv('orders.csv')

pricey_shoes = orders.groupby('shoe_type').price.max().reset_index()
print(pricey_shoes)
print(type(pricey_shoes))
```

Расчет агрегатных функций III

Иногда операция, которую вы хотите выполнить, сложнее среднего или подсчета. В этих случаях вы можете использовать метод `apply` и лямбда-функции, как мы это делали для операций с отдельными столбцами. Обратите внимание, что входными данными нашей лямбда-функции всегда будет список значений.

Прекрасный пример этого - вычисление процентилей. Предположим, у нас есть `DataFrame` информации о сотрудниках с именем `df`, который имеет следующие столбцы:

- `id`: идентификационный номер сотрудника
- `name`: имя сотрудника
- `заработная плата`: почасовая оплата работника
- `категория`: тип работы, которую выполняет сотрудник

Наши данные могут выглядеть примерно так:

id	name	wage	category
10131	Sarah Carney	39	product
14189	Heather Carey	17	design
15004	Gary Mercado	33	marketing
11204	Cora Copaz	27	design
...			

Если мы хотим рассчитать 75-й процентиль (то есть точку, в которой 75% сотрудников имеют более низкую заработную плату, а 25% - более высокую заработную плату) для каждой категории, мы можем использовать следующую комбинацию применения и лямбда-функции:

```
# np.percentile can calculate any percentile over an array of values
high_earners = df.groupby('category').wage
                .apply(lambda x: np.percentile(x, 75))
                .reset_index()
```

Результат high_earners может выглядеть так:

	category	wage
0	design	23
1	marketing	35
2	product	48

Задание

1. Еще раз вернемся к данным с ShoeFly.com. Наши специалисты по маркетингу говорят, что очень важно иметь доступную по доступной цене обувь для каждого цвета обуви, которую мы продаем.

Давайте рассчитаем 25-й процентиль цены на обувь для каждого цвета shoe_color, чтобы помочь маркетингу решить, достаточно ли у нас дешевой обуви на распродаже. Сохраните данные в переменной cheap_shoes.

Примечание: обязательно используйте reset_index () в конце вашего запроса, чтобы cheap_shoes был DataFrame.

Расчет агрегатных функций IV

Иногда нам нужно сгруппировать по более чем одному столбцу. Мы можем легко сделать это, передав список имен столбцов в метод `groupby`.

Представьте, что у нас есть сеть магазинов и есть данные о количестве продаж в разных местах в разные дни:

Location	Date	Day of Week	Total Sales
West Village	February 1	W	400
West Village	February 2	Th	450
Chelsea	February 1	W	375
Chelsea	February 2	Th	390

Мы подозреваем, что продажи в разных местах в разные дни недели различаются. Чтобы проверить эту гипотезу, мы могли бы рассчитать средние продажи для каждого магазина в каждый день недели за несколько месяцев. Код будет выглядеть так:

```
df.groupby(['Location', 'Day of Week'])['Total Sales'].mean().reset_index()
```

Location	Day of Week	Total Sales
Chelsea	M	402.50
Chelsea	Tu	422.75
Chelsea	W	452.00
...		
West Village	M	390
West Village	Tu	400
...		

Задание

1. На ShoeFly.com наша команда по закупкам считает, что в этом году особенно популярны определенные комбинации `shoe_type` / `shoe_color` (например, синие балетки очень популярны в Париже).

Создайте DataFrame с общим количеством обуви для каждой купленной комбинации `shoe_type` / `shoe_color`. Сохраните его в переменной `shoe_counts`.

Вы должны иметь возможность сделать это с помощью `groupby` и `count ()`.

Примечание. Когда мы используем `count ()`, не имеет значения, в каком столбце мы выполняем вычисления. В этом примере вы должны использовать `id`, но мы получили бы тот же ответ, если бы использовали `shoe_type` или `last_name`.

Выведите результат в консоль

